Practical class 1: implementing a deep network

In this session, you will use Tensorflow and Keras to implement neural networks with large amount of layers. For practical reasons, we will use Google Colab (except for the last network).

I A quick warm-up with TensorFlow and Keras

You will first reproduce your multi-layer network with these libraries.

➔ It is strongly recommended to write the different parts of the code in separate cells to avoid having to re-run certain parts or re-download a dataset.

Start with libraries imports:

```
import tensorflow as tf
from tensorflow import keras

import numpy as np
import matplotlib.pyplot as plt
```

Then load MNIST dataset:

```
# import dataset
 (img_train, label_train),(img_test,label_test) = keras.datasets.mnist.load_data()

# conversion of int values into float
img_train=img_train/255.0
img_test=img_test/255.0

# conversion of labels into output vectors using to_categorical (with 10 classes)
output_train=keras.utils.to_categorical(label_train, num_classes=10)
output_test=keras.utils.to_categorical(label_test, num_classes=10)

# we check the size of our dataset
print(img_train.shape)
print(output_train.shape)

# display a sample of images
for i in range(10):
  plt.subplot(2,5,i+1)
  plt.imshow(img_train[i], cmap='gray')

plt.show()
```

Then, build the network. Note the *Flatten* layer to convert images into input vectors. We then add two *Dense* layers with respectively 20 and 10 neurons, with a sigmoid activation function:

```
model = keras.Sequential( )
model.add(keras.layers.Input(shape=(28, 28)))
model.add(keras.layers.Flatten())
model.add(keras.layers.Dense(20, activation='sigmoid'))
model.add(keras.layers.Dense(10, activation='sigmoid'))

# display structure
model.summary()
```

You can observe the network's structure and number of trainable parameters

Compile the network using the loss function *MeansSquaredError* and gradient descent (*SGD*) with a learning rate of 0.05:

```
loss=keras.losses.MeanSquaredError()
optim=keras.optimizers.SGD(learning_rate=0.05)

model.compile(loss=loss, optimizer=optim, metrics=["accuracy"])
```

Finally, train your network with function *fit*. We will perform 20 epochs with a batch of 4:

```
model.fit(img_train, output_train, batch_size=4, epochs=20, shuffle=True, verbose=2)
```

We can notice that results are slightly lower than with our Java implementation: the use of multiple simultaneous samples (batch) increases training speed, but at the expense of efficiency.

Evaluate your network on test dataset:

```
model.evaluate(img_test,output_test, batch_size=4, verbose=2)
```

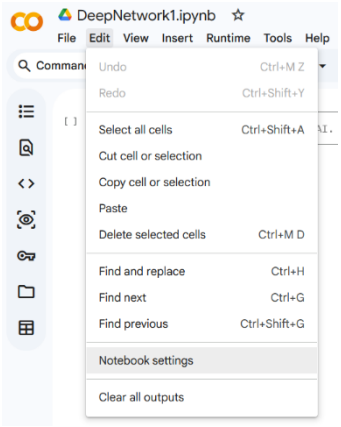
You can also use your network to calculate predictions:

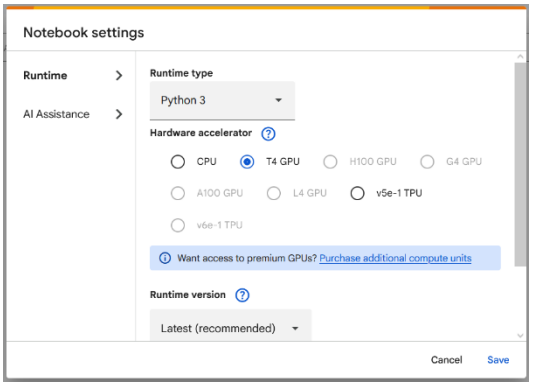
```
for i in range(50):
  output=model.predict(img_test[i:i+1,:,:])
  print(label_test[i]," , ",output.argmax())
```

II A first deep network

Let's get started with the serious business: we will build a deep convolutional network to improve image recognition of images from dataset *MNIST fashion*. Stop the current session (Runtime -> Manage sessions -> delete active session) and create a new Colab document that you will call *DeepNetwork1*.

➔ To allow network training within a reasonable amount of time, we will use GPU provided by Google Colab: Edit -> Notebook settings-> T4-GPU





➔ Do not forget to deactivate the GPU and stop active sessions at the end to be sure to release resources (and avoid resource ban from Colab).

Run the following code to check the GPU:

```
import tensorflow as tf
device_name = tf.test.gpu_device_name()
if device_name != '/device:GPU:0':
  raise SystemError('GPU device not found')
print('Found GPU at: {}'.format(device_name))
```

```
Found GPU at: /device:GPU:0
```

If the message appears, the GPU is used.

Add the libraries import in a Colab cell:

```
from tensorflow import keras
import matplotlib.pyplot as plt
import numpy as np
```

We now load the dataset and convert types.

```
import tensorflow_datasets as tfds

dataset = keras.datasets.fashion_mnist
(img_train, label_train),(img_test,label_test) = dataset.load_data()

# conversion of int images into float
img_train=img_train/255.0
img_test=img_test/255.0

output_train=keras.utils.to_categorical(label_train, num_classes=10)
output_test=keras.utils.to_categorical(label_test, num_classes=10)

print(img_train.shape)
print(img_test.shape)
```

```
(60000, 28, 28)
(10000, 28, 28)
```

We obtain respectively 60 000 and 10 000 2D images of 28x28 pixels.
Problem: a convolutive layer uses images with a depth. Color images already have 3 channels, but it is not the case of greyscale images. We thus add the following instructions to add a third dimension to our images (then display the shape of these matrices) :

```
img_train2 = np.expand_dims(img_train, axis=-1)
img_test2  = np.expand_dims(img_test, axis=-1)
```

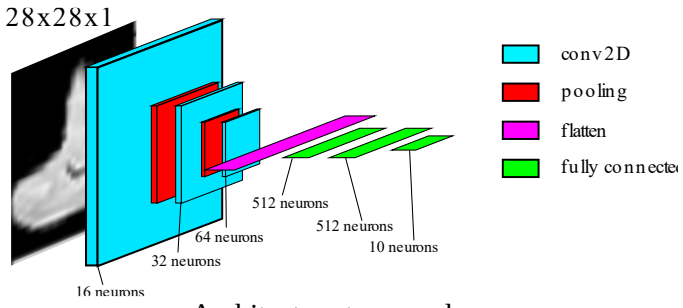
```
(60000, 28, 28, 1)
(10000, 28, 28, 1)
```

Display a sample of images (using the 2D images):

```
for i in range(10):
  plt.subplot(2,5,i+1)
  plt.imshow(img_train[i], cmap='gray')

plt.show()
```

Then, build a sequential network according to the following architecture:



Architecture to reproduce

Observe the number of trainable parameters. The size of this network remains sufficiently reasonable for a training with a MNIST dataset.

Compile your network with loss function *CategoricalCrossEntropy* and an optimization function *Adam* with a learning rate of 0.001. You will display the efficiency (‘accuracy’) to follow the training evolution.

Perform the network training on the training dataset, with 20 epochs and a batch size of 16. The learning process lasts for around 5 minutes, use this opportunity to take a breath…

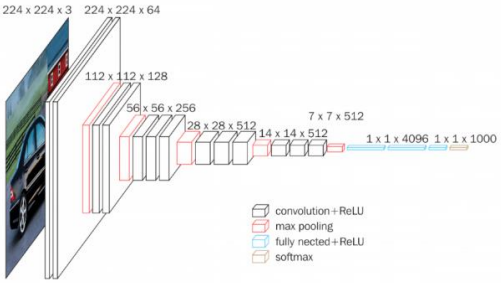
Finally, measure the efficiency of your network with the test dataset. Note the success rate of your network.

```
model.evaluate(img_test2, output_test, batch_size=16, verbose=2)
```

III Let's continue with transfer learning

The *transfer learning* allows us to construct and train networks this large number of layers with a limited dataset and within a reasonable amount of time. We will use a famous architecture for image recognition: the VGG16

Create a new Colab document, called *DeepNetwork2* (don't forget to release GPU and stop previous session)



L'architecture VGG16

This network was proposed at the l'ILSVRC 2014, and trained on the ImageNet dataset, gathering 14 million of images from 1000 classes (training on a farm of GPU NVIDIA Titan Black during multiple days).

We will get trained layers of this network (with trained weights), freeze these layers and replace the fully-connected part with a new network. Our network will be trained to recognize images of cats and dogs.

This time, we will use a dataset available online, that we will get in Colab with the following command (use a specific cell to download it only once):

```
!wget --no-check-certificate \
    https://huggingface.co/datasets/pantelism/cats-vs-dogs/resolve/main/dogs-vs-cats-
subset.zip \
    -O /tmp/cats_and_dogs_filtered.zip
```

Unzip the archive, you can notice paths of directories (can change if you work on your own local computer). Files are visible in the explorer, available on the left of the Colab interface.

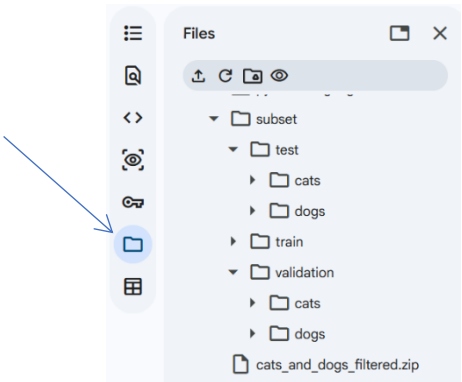
```
import os
import zipfile

local_zip = '/tmp/cats_and_dogs_filtered.zip'
zip_ref = zipfile.ZipFile(local_zip, 'r')
zip_ref.extractall('/tmp')
zip_ref.close()

train_dir = '/tmp/subset/train'
test_dir = '/tmp/subset/validation'
```

Once extracted, the directory structure looks like the following:

```
--- subset
        |
        |----------train
        |             |
        |             |---------- cats
        |             |               |--------liste d'images .jpg
        |             |
        |             |----------dogs
        |                           |--------liste d'images .jpg
        |----------validation
                      |
                      |---------- cats
                      |               |--------liste d'images .jpg
                      |
                      |----------dogs
                                      |--------liste d'images .jpg
```



This directory structure is made on purpose: it can be exploited to generate a training dataset and an evaluation dataset, using a specific Keras object: the ImageDataGenerator

This object will browse the directory structure to feed the network with training or test samples, taking the directory and subdirectory into account to define the label of each image. In a new cell, create two generators to generate two datasets of 224x224x3 images (for training and for learning)

```
trainDataGenerator = keras.preprocessing.image.ImageDataGenerator(rescale=1./255)
trainData = trainDataGenerator.flow_from_directory(directory=train_dir, target_size=(224,224))

testDataGenerator = keras.preprocessing.image.ImageDataGenerator(rescale=1./255)
testData = testDataGenerator.flow_from_directory(directory=test_dir, target_size=(224,224))
```

```
Found 2000 images belonging to 2 classes.
Found 1000 images belonging to 2 classes.
```

We can now load the pre-trained VGG16 network, with weights obtained after training with dataset *ImageNet*. Keras proposes function to load most common and efficient networks (list of available networks on https://keras.io/api/applications/). Observe the number of trainable parameters.

```
VGG16= keras.applications.VGG16(weights="imagenet")

VGG16.summary()
```

It is clear that we will not be able to train a network of this size ourselves. We can, however, use this network to make predictions about our images. Get the list of available images:

```
train_cats_files = os.listdir(train_dir+"/cats")
print(train_cats_files[:10])

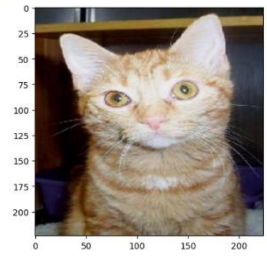
train_dogs_files = os.listdir(train_dir+"/dogs")
print(train_dogs_files[:10])
```

Then get a single image (you can change the index and directory ‘dogs’ or ‘cats’) and perform a prediction:

```
index=3
img = keras.preprocessing.image.load_img(
                                    test_dir+"/cats/"+test_cats_files[index],target_size=(224,224))
img = np.asarray(img)
plt.imshow(img)
img = np.expand_dims(img, axis=0)

prediction=VGG16.predict(img)
prediction.argmax()
```

```
1/1 [==============================] - 0s 52ms/step
281
```



The list of 1000 classes of ImageNet is available on https://gist.github.com/yrevar/942d3a0ac09ec9e5eb3a (Class 281 is ‘tabby cat’).

The network was thus already trained on images of cats and dogs, which will facilitate the transfer learning for our recognition task cats VS dogs.

Keras allows to only load convolutional layers of a network with the parameter ‘include_top’ that we will set to False. Load the network VGG16 again, but without the fully-connected part:

```
base_VGG16= keras.applications.VGG16(weights="imagenet", include_top=False, input_shape=(224,224,3))

base_VGG16.summary()
```

Note where the network is cut. Also, with the cut network, images of 150x150px can be used (we will continue to use images 224x224 in this session).

We will use a feature extraction, and thus, freeze all convolutional layers, by setting the parameter 'trainable' of the network to False:

```
base_VGG16.trainable=False

base_VGG16.summary()
```

All network parameters are now 'Non-trainable'.

Note: each layer of a network has its own parameter 'trainable', making possible to only freeze a part of the network (this is useful, for example, for a partial fine-tuning).

We now replace the missing fully-connected part. Add a hidden layer of 50 neurons with ReLU activation function, and an output layer of 2 neurons (2 classes) with a SoftMax activation layer.

The two approaches, sequential and functional, are possible to form the complete network. We will use the sequential approach.

Start with a new global network, then add the cut VGG16 network.

```
model=keras.models.Sequential()
model.add(base_VGG16)
```

Then add the two fully connected layers.

Display the network structure (summary). You can notice that networks can be constructed with other networks, which can be useful for more complex networks such as GANs.

Compile the global network using the loss function CategoricalCrossentropy and learning optimizer Adam (with a learning rate of 0.001)

The network can then be trained and evaluated using our dataset generators (labels are provided by these generators):

```
model.fit(trainData, epochs=5, batch_size=32)
```

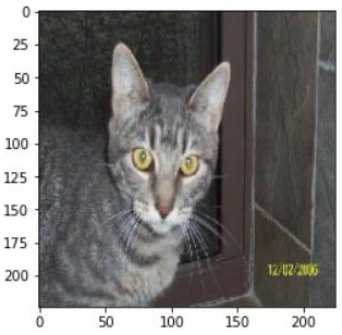
```
model.evaluate(testData, batch_size=32, verbose=2)
```

Note the efficiency of your network. Not that bad for a network trained with only 2000 images !

You can then use your network to recognize images (such as previously). The following code can write the response of the network.

```
index=100
img = keras.preprocessing.image.load_img(test_dir+"/cats/"+test_cats_files[index],tar-
get_size=(224,224))
img = np.asarray(img)
plt.imshow(img)
img = np.expand_dims(img, axis=0)
prediction=model.predict(img)
if prediction.argmax()==0:
  print("cat")
else :
  print("dog")
```

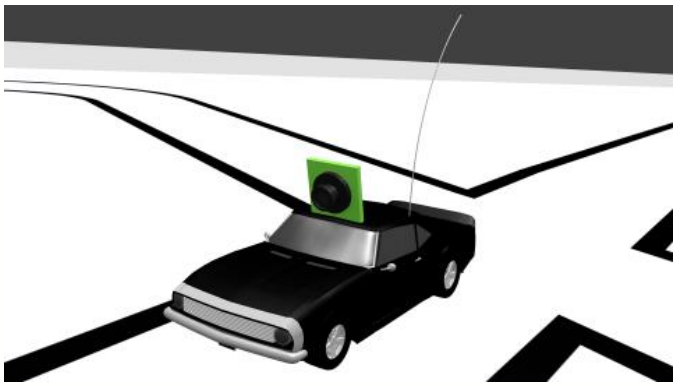
```
cat
```

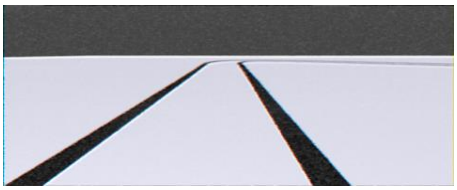


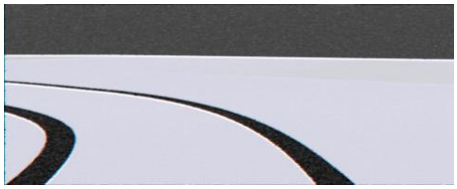
(Do not forget to release the GPU and close the active session)

IV Drive a RC car

In this exercice, we will train a convolutional network to drive a RC car on a road made with two lines of black tape. The use of convolutional networks enables the use of larger images than fully-connected networks.







The exercise is divided into two parts: the training part will use Google Colab and its GPU. The exploitation part will use a local Jupyter-Notebook interacting with a 3D simulator.

First, download the archive car_dataset.zip from the usual Github. The archive contains color images of 320x240 pixels distributed in 13 directories. Each corresponding to a turn movement (0 for full right turn, 6 for forward, 12 for full left turn). These images where obtained by driving the simulated RC car during two laps.

On Google Colab, create a new document called *autonomousCar* and set a GPU to this session. Add the following code in a new cell and run it. A button appears, use it to import the zip archive:

```
from google.colab import files
uploaded = files.upload()
```

Then, extract the archive in /tmp:

```
import os
import zipfile
local_zip = '/content/car_dataset.zip'
zip_ref = zipfile.ZipFile(local_zip, 'r')
zip_ref.extractall('/tmp')
zip_ref.close()
```

Now, load usual libraries:

```
import tensorflow as tf

device_name = tf.test.gpu_device_name()
if device_name != '/device:GPU:0':
  raise SystemError('GPU device not found')
print('Found GPU at: {}'.format(device_name))

from tensorflow import keras
import matplotlib.pyplot as plt
import numpy as np
```

Define a unique dataset generator for training:

```
train_dir = '/tmp/dataset/train'

trainDataGenerator = keras.preprocessing.image.ImageDataGenerator(rescale=1./255)
trainData = trainDataGenerator.flow_from_directory(directory=train_dir, tar-
get_size=(240,320))
```

The generator should find 3335 images in 13 classes.

Then, construct your network, following this architecture (/!\ the input size is height x width x depth):

- Convolutional layer of 16 neurons, kernel of 5x5, ReLU
- Pooling
- Convolutional layer of 32 neurons, kernel of 3x3, ReLU
- Pooling
- Convolutional layer of 32 neurons, kernel of 3x3, ReLU
- Flatten
- Fully-connected layer of 24 neurons, ReLU
- Fully-connected layer of 24 neurons, ReLU
- Fully-connected layer of 13 neurons, SoftMax

Compile the network with CategoricalCrossentropy loss, Adam optimizer with learning rate of 0.001, and train on 70 epochs with a batch size of 16.

The network is now ready. We need to export it in order to run it on local machine. In two separated cells: save the network as a h5 file:

```
model.save_weights("model_RC_car.weights.h5")
```

And download the h5 file locally:

```
files.download("model_RC_car.weights.h5")
```

We can now release GPU and close all active sessions. For the next part, it is recommended to copy the code of your network architecture and paste it in a notepad.

From now, we will work on a local Jupyter-Notebook. Start a local session (*python -m notebook* or *jupyter lab*)) and create a new Jupyter file. We will reconstruct the Colab network locally and connect it with a 3D simulator. First, download the simulator archive *car_simulation_OS.zip* (Linux or windows depending on your OS) with the following link:
https://filesender.renater.fr/?s=download&token=85993ba8-c174-447d-a686-036e87edf814
and unzip it in your work directory.

The simulation is based on Blender Game Engine and communicates with external programs through two text files:

- First, the external program writes a direction command in file *orientation.txt*
- The simulator waits for a command in orientation.txt, then clears the file.
- Command is simulated, then the simulator takes a picture of the camera recorded in folder *img*. The frame number is written in file *frame.txt*
- The external program waits for a frame number in frame.txt, then erase file content. It can then use last image frame and send a new command in *orientation.txt*

Note: the sequence of image can be used to generate a video!

In your Jupyter file, start by importing libraries:

```
import tensorflow as tf
from tensorflow import keras
import numpy as np
import time
```

Then, construct your network. Attention: it must be exactly the same than on Colab, or weight import will not work.

You can then load weights with

```
from tensorflow.keras.models import load_model

model.load_weights("PATH/TO/MODEL/model_RC_car.weights.h5")
```

Your model is now ready! We can now define functions to communicate with the simulator: the first function waits for a value in frame.txt, then clears the file. The second function waits until orientation.txt is empty and write new command.

```
frame_file = "/PATH/TO/FILE/frame.txt"
orientation_file = "PATH/TO/FILE/orientation.txt"

def read_frame(file):
    while True:
        # open frame.txt
        with open(file, "r") as f:
            frame = f.read().strip()
        # wait while file is empty
        if frame != "":
            # clear file
            with open(file, "w") as f:
                f.write("")
            return frame

        time.sleep(0.1)

def wait_write(file):
    while True:
        # open frame.txt
        with open(file, "r") as f:
            frame = f.read().strip()
        # wait until file is empty
        if frame == "":
            return

        time.sleep(0.1)
```

And finally, the main control loop: the system read frame number and get the corresponding image. The image is converted as an input for the neural network, that provide a

```
# List of orientations in degrees
angles = [-160, -120, 90, 120, 160, -90, -60, -30, -15, 0, 15, 30, 60]

# write first command
with open(orientation_file, "w") as f:
    f.write(str(8))

# Main loop
while True:
    # wait new frame
    frame = read_frame(frame_file)

    try:
        frame_number = int(frame)
    except ValueError:
        print("invalid number:", frame)
        continue

    # Load image
    image_path =  f"PATH/TO/IMAGES/img/frame{frame_number}.png"
    img=keras.preprocessing.image.load_img(image_path,color_mode="rgb",target_size=(240,320))

    # convert image
    img_array = keras.preprocessing.image.img_to_array(img)
    img_array = img_array / 255.0
    img_array = np.expand_dims(img_array, axis=0)

    # Prediict with network
    prediction = model.predict(img_array, verbose=0)
    orientation = np.argmax(prediction)
    angle=angles[orientation]
    print("Predicted orientation: ", orientation," , angle=", angle)

    # Synchronize with simulator and write command
    wait_write(orientation_file)
    with open(orientation_file, "w") as f:
        f.write(str(angle))
```

You can now try your autonomous RC car: remove images from *img* folder (if any), remove the content of file *frame.txt*. Then, run the simulator (*simulation_Linux* or *simulation_windows.exe*), and when the environment appears, run the Jupyter cell with the main control loop. Will your car drive correctly?